

A two-way APRS iGate that will only ever transmit to my whitelist

KD3CCO

Texting a handheld from a phone, where the cell network does not reach

I wanted to text my wife from a radio where there is no cell coverage

An SMS leaves a phone, crosses the internet, reaches a gateway I own, and comes out of a radio.

The phone end is an ordinary text message. **The radio end needs no cell service at all** — and that last leg is the whole project.

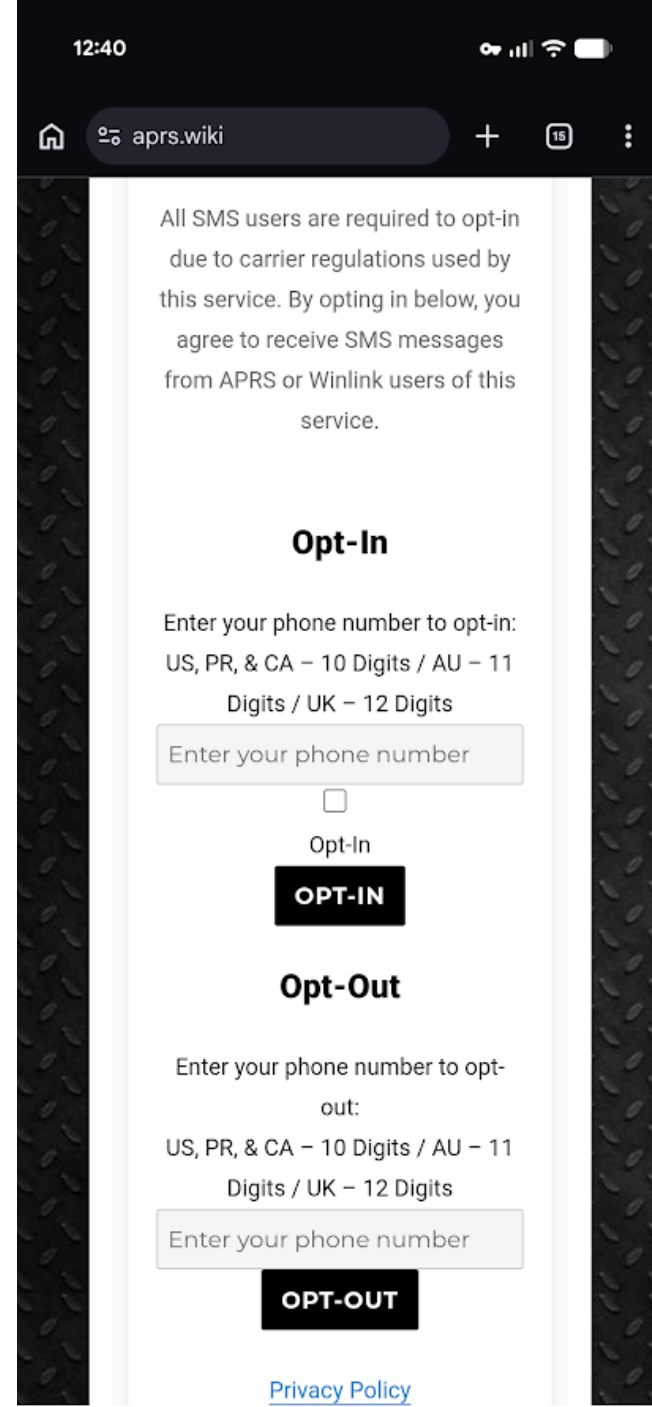


Half of what I wanted, you can have tonight, without building anything

Radio to phone already works — for everyone, for free. NA7Q runs an APRS-to-SMS bridge, documented at aprs.wiki. It answers to `SMS` on the air, and to `866-352-4096` from a phone.

1. **Opt the number in** on this form. Ten digits, press the button. It is a carrier requirement — skip it and your messages vanish with **no error anywhere**.
2. **From the radio**, send an APRS message to `SMS` reading `@2125550123 your text`.

No hardware you do not already own. Five minutes.



The screenshot shows a mobile browser interface for the website aprs.wiki. At the top, the status bar shows the time 12:40 and various icons. The browser address bar displays the URL. The main content area has a dark background with a textured pattern. It contains two sections: 'Opt-In' and 'Opt-Out'. Each section has a heading, a paragraph explaining the requirement, a text input field for the phone number, a checkbox, and a button labeled 'OPT-IN' or 'OPT-OUT'. A 'Privacy Policy' link is at the bottom.

12:40

aprs.wiki

All SMS users are required to opt-in due to carrier regulations used by this service. By opting in below, you agree to receive SMS messages from APRS or Winlink users of this service.

Opt-In

Enter your phone number to opt-in:
US, PR, & CA – 10 Digits / AU – 11 Digits / UK – 12 Digits

☐
Opt-In
OPT-IN

Opt-Out

Enter your phone number to opt-out:
US, PR, & CA – 10 Digits / AU – 11 Digits / UK – 12 Digits

OPT-OUT

[Privacy Policy](#)

Then set up an alias, so the number never goes on the air again

APRS is transmitted in clear text and archived publicly. Address a phone by its raw number and you publish that number, beside your callsign, on every single message.

Send one message to SMS :

```
#alias #add wife 5705550123
```

It answers "**Alias Added.**" From then on you address @wife , and the number is never transmitted again.

Send that same command from the phone instead, and it never touches RF at all.



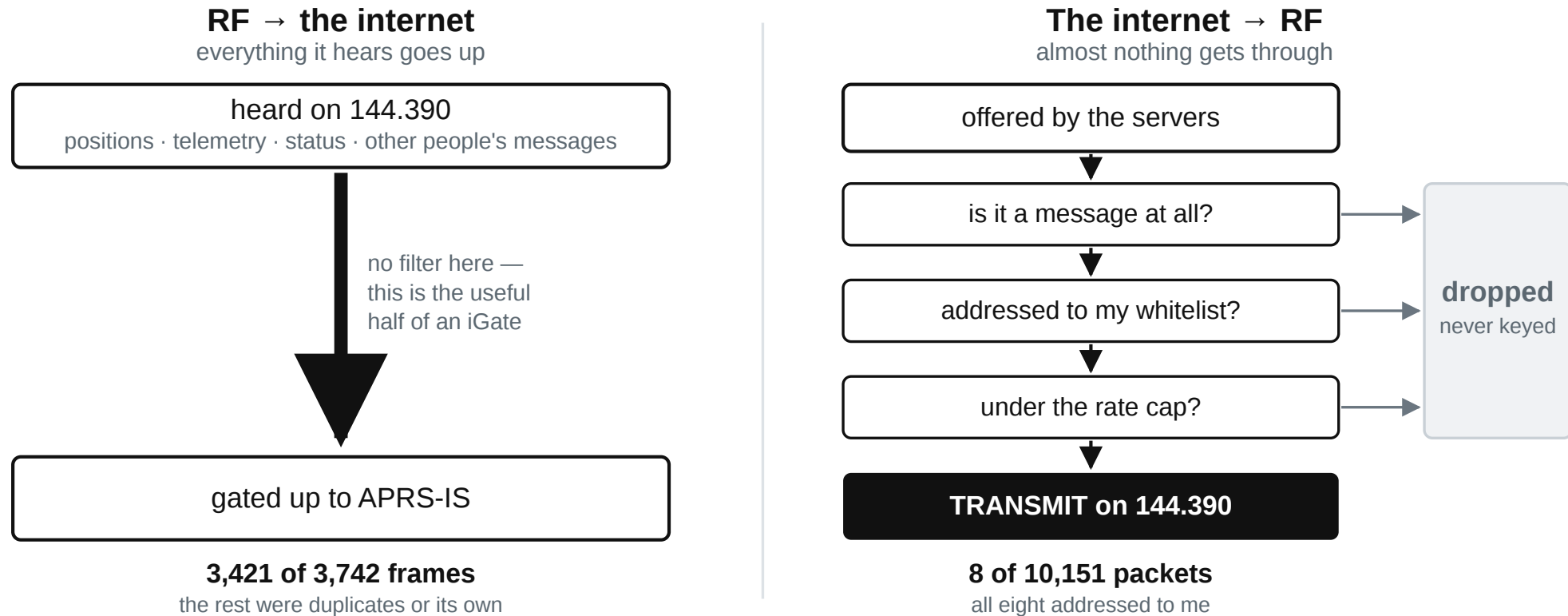
Getting a message back to the radio is the half you need to build yourself

That bridge will happily carry my text to a phone. Nothing sends one back to me.

Most iGates are **receive only**: they listen on 144.390 and pass what they hear up to the internet. Nothing goes back down. Going the other way means running **a transmitter that traffic from the internet can key** — which is the part worth being careful about, and the reason I built my own rather than asking to enable it on someone else's.

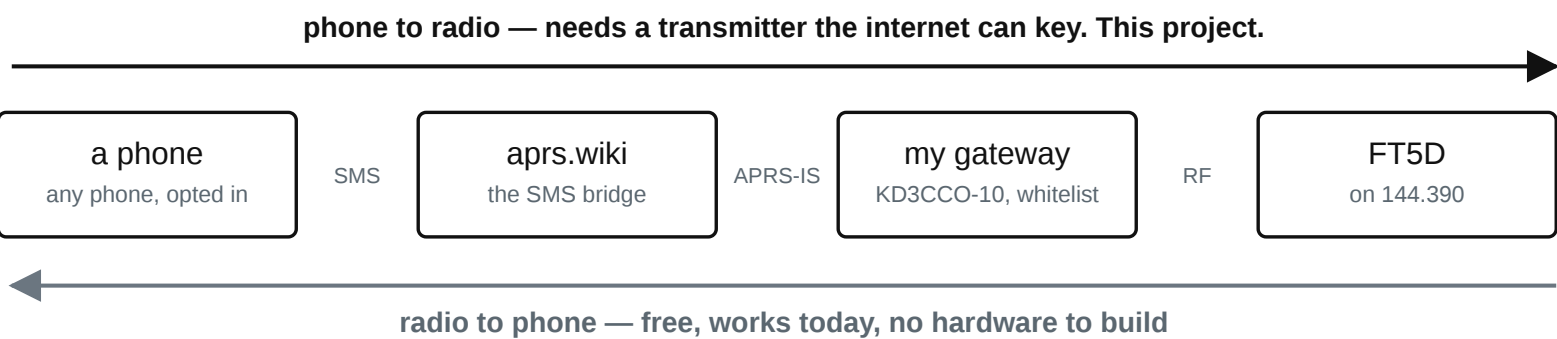
That is the whole project: the return leg.

For my iGate, only APRS messages addressed to a callsign on my whitelist are ever transmitted

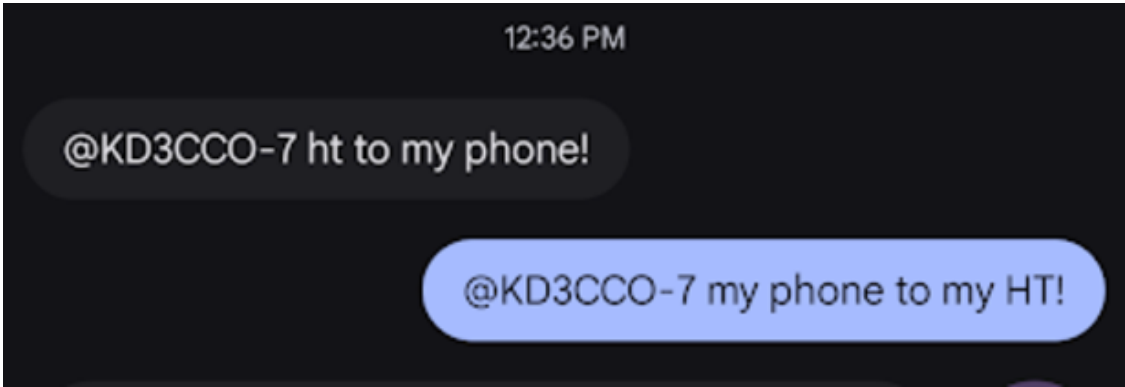


Measured over thirty hours. From every other operator's point of view this station is receive-only, so its beacon says exactly that rather than advertising a relay nobody else can use.

Both directions work, end to end, through the public SMS bridge



sent from the radio



both messages, on the phone



the reply, back on the radio

Four lines of configuration do the work that matters

```
IGFILTER  g/KD3CCO*      ALSO send me messages for these calls
FILTER IG  0 g/KD3CCO*    what I will put ON THE AIR – the whitelist
IGMSP      0              no courtesy exceptions to it
IGTXLIMIT  6 10           at most 6 packets a minute, 10 in five
```

IGFILTER widens what arrives; it does not narrow it. APRS-IS already sends an iGate traffic involving stations it has recently gated up — which is most of what you see arriving and being dropped. This *adds* a subscription: send me messages for my calls **whether or not I have heard that station lately**. Without it the downlink only reaches somebody already in earshot, which is the opposite of the point.

FILTER IG 0 is the gate. It decides what reaches the transmitter. That one is the guarantee.

IGTXLIMIT is a hard ceiling whatever the rest of the config says — and Dire Wolf **drops** packets over it rather than queueing them, which is the right behavior on a shared national channel.

IGMSP 0 is the one I would have missed. Dire Wolf has a courtesy feature that transmits a **message sender's position** after relaying their message — regardless of any filter. I watched it put the SMS gateway's own beacon on the air.

A whitelist with a well-intentioned exception is not a whitelist.

The filter is visible while it works, not just in the config file

A phone in the shack, watching a round trip as it happens. **IS GATED** in green is the one packet the whitelist let onto the air; five seconds later the handheld's acknowledgement comes back on RF.

Over thirty hours the servers offered this station **10,151 packets. Eight reached the air** — every one a message to a whitelisted call. The other transmissions were its own beacons.

Every line is labeled with the direction it went, which is how three separate faults were eventually cornered.

```
1:06 192.168.68.66:8080 + 15 i
iGate Monitor · aprs-igate
state running call KD3CCO-10
whitelist KD3CCO*, KD3CCP*, W3EDP* feed live
▶ Station and whitelist
13:06:27 IS DROP
NV8P-1>APRSAR,TCPIP*,qAC,THIRD:=4115.54N/08143.54W#0-
1/000/A=000892Testing SarTrack System[:Buzzard Digi
13:06:27 IS DROP
KD3SU-
10>APMI04,TCPIP*,qAS,KD3SU:@271705z3928.99NI07737.90W&iGate
600ft HAAT U=11.7V<0x20>
13:06:29 RF RX
W3NEP-2>APMI06,K3DNA,W03T-1,WIDE2*:>146.610-
R*111111z4110.57N/07521.73Wr T082 R50 LCARES MG1st W
13:06:29 RF->IS UP
W3NEP-2>APMI06,K3DNA,W03T-1,WIDE2*,qAR,KD3CCO-10:>146.610-
R*111111z4110.57N/07521.73Wr T082 R50 LCARES MG1st W
Status Report, SQ3PLX http://microsat.com.pl/
146.610-R*111111z4110.57N/07521.73Wr T082 R50 LCARES
MG1st W
13:06:42 IS DROP
KW3DX-
1>APMI06,TCPIP*,qAC,T2BC:T#240,190,025,010,076,000,00000000
13:06:42 IS GATED
SMS>APOSMS,TCPIP*,qAC,WA7BF::KD3CCO-7 :@self my phone to my
HT!{17986
13:06:47 RF RX
KD3CCO-7>APY05D,WIDE1-1,WIDE2-1::SMS :ack17986<0x0d>
13:06:47 RF->IS UP
KD3CCO-7>APY05D,WIDE1-1,WIDE2-1,qAR,KD3CCO-10::SMS
:ack17986
The APRS protocol specification says nothing about a
possible carriage return after the
message id. Adding CR might prevent proper
interoperability with with other applications.
"KD3CCO-7" ACKnowledged message number "17986" from
"SMS", Yaesu FT5D series
```

It is a box you plug in and forget



A Raspberry Pi 3B+, a \$35 USB sound-card interface, and a mobile radio. No keyboard, no monitor, no screen.

It now tests itself every four hours and says which link failed

```
step 1 OK: APRS-IS accepted it  
step 2 OK: the server routed it back to this gateway  
step 3 OK: the gateway transmitted it  
step 4 OK: KD3CCO-7 acknowledged it over RF
```

PASS — a message from the internet reached KD3CCO-7 and was acknowledged.

A beacon proves the transmitter keys. **Only an acknowledged message proves the station can do the thing it exists for** — and this one had spent hours beaoning happily while unable to deliver anything.

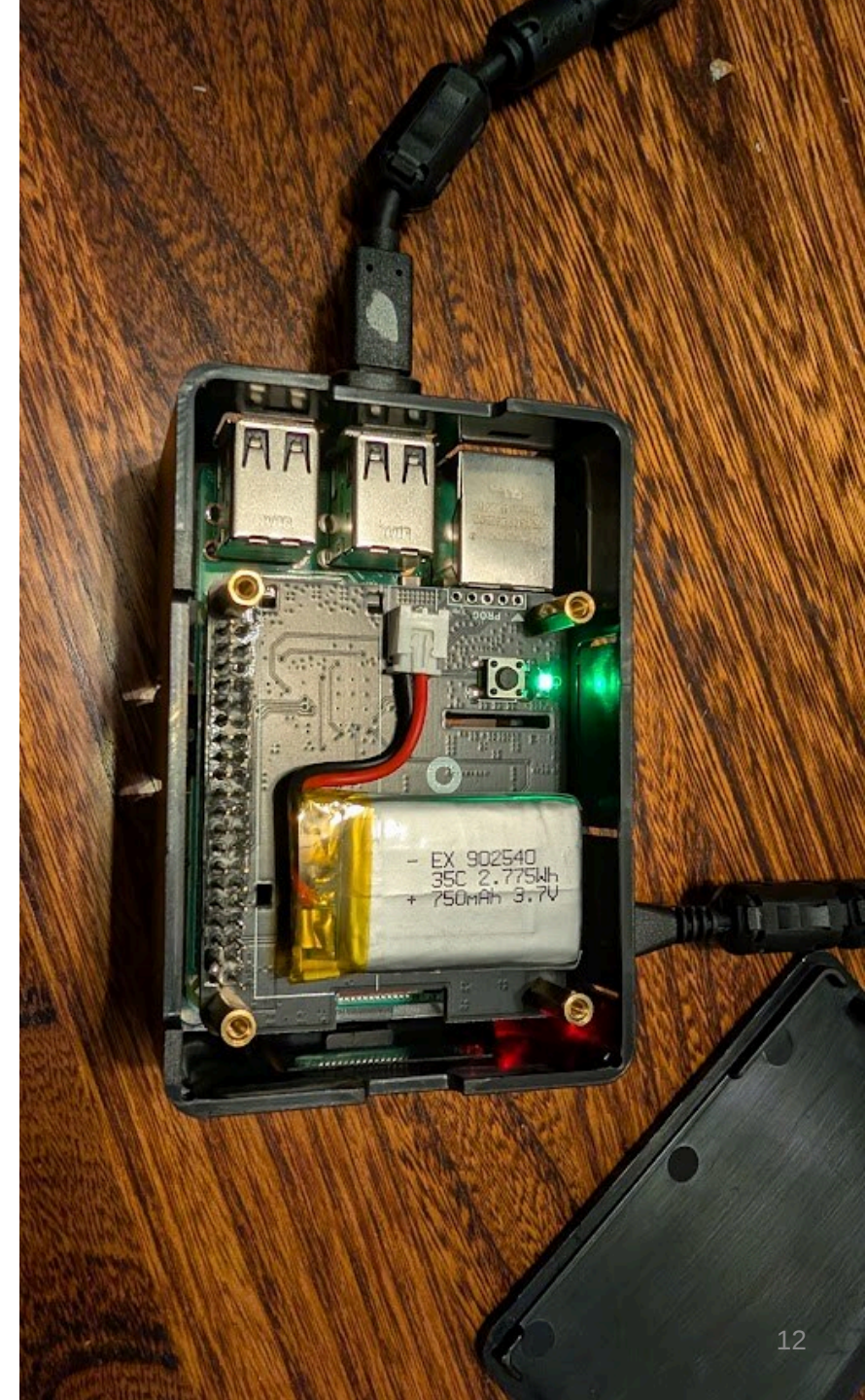
It shuts itself down now, when the power goes

A UPS HAT on the GPIO header: a lithium cell that carries the Pi on its own for a while, and a battery-backed clock.

The battery is not the protection — the shutdown is. A UPS that keeps the Pi running and then dies mid-write corrupts the card exactly as a pulled plug does. Thirty seconds after mains is lost it powers off, with most of the cell still unused.

It does not keep the station on the air. The radio is dark the moment mains goes. This only means the gateway goes down cleanly instead of badly.

The clock on it is the second win: until now this station came up with the wrong time and timestamped every packet it gated until the network corrected it.



The three faults that cost the most were none of them software

An adapter cable in backwards. Three conductors into a four-conductor socket grounded the PTT line, so the radio keyed a dead carrier until its own timeout. Receive was perfect and every log said the messages were sent.

RF from its own antenna, knocking the sound card off the USB bus — one reset every other transmission.

An antenna that was not one. A mobile whip indoors, sold as needing no ground plane, which still needed far more counterpoise than I gave it: the coax braid ended up doing the radiating. Eight decibels more power bought nothing.

All three were fixed by a slim jim hung outside, a choke at the feedpoint, and a cable the right way round. **I guess I really am an amateur at this radio stuff.**



What it takes, if you want one

The gateway — a Raspberry Pi 3B+, a microSD card, a 5 V supply.

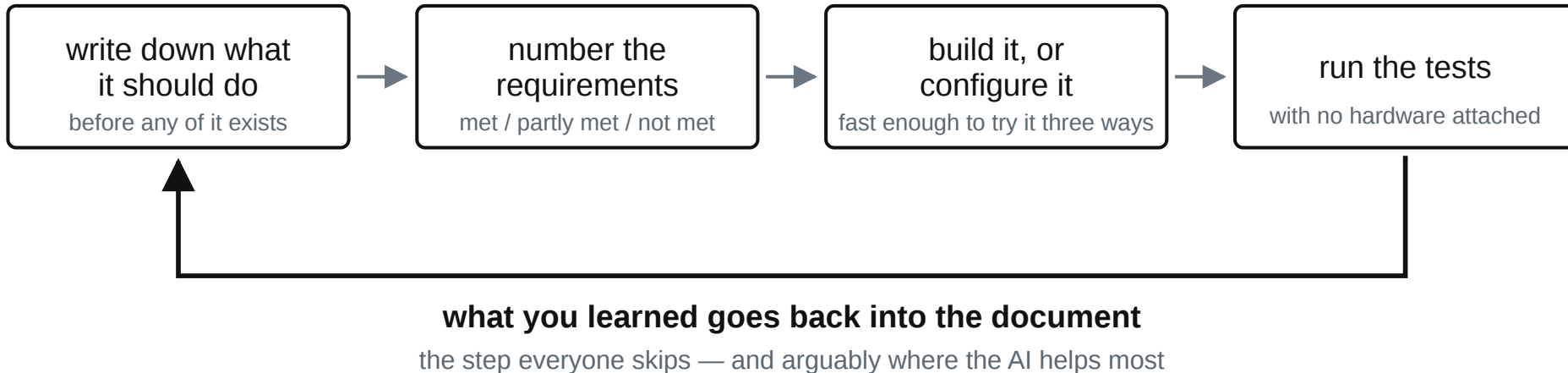
The radio interface — a Digirig Lite, about \$35, and the cable for your radio. That cable is the single most error-prone item here; mine was plugged in backwards for four hours.

A 2 m radio that will sit on 144.390 and accept external PTT. An FT-2900R here; a VX-6R and a Radtel both work on the same profile.

An antenna that is actually an antenna — a slim jim outside, with a choke at the feedpoint. This mattered more than any software I wrote.

No CAT cable, no GPS, no display, no keyboard, no monitor — ever. **Try the whole thing in Docker on a laptop first**, before buying any of it.

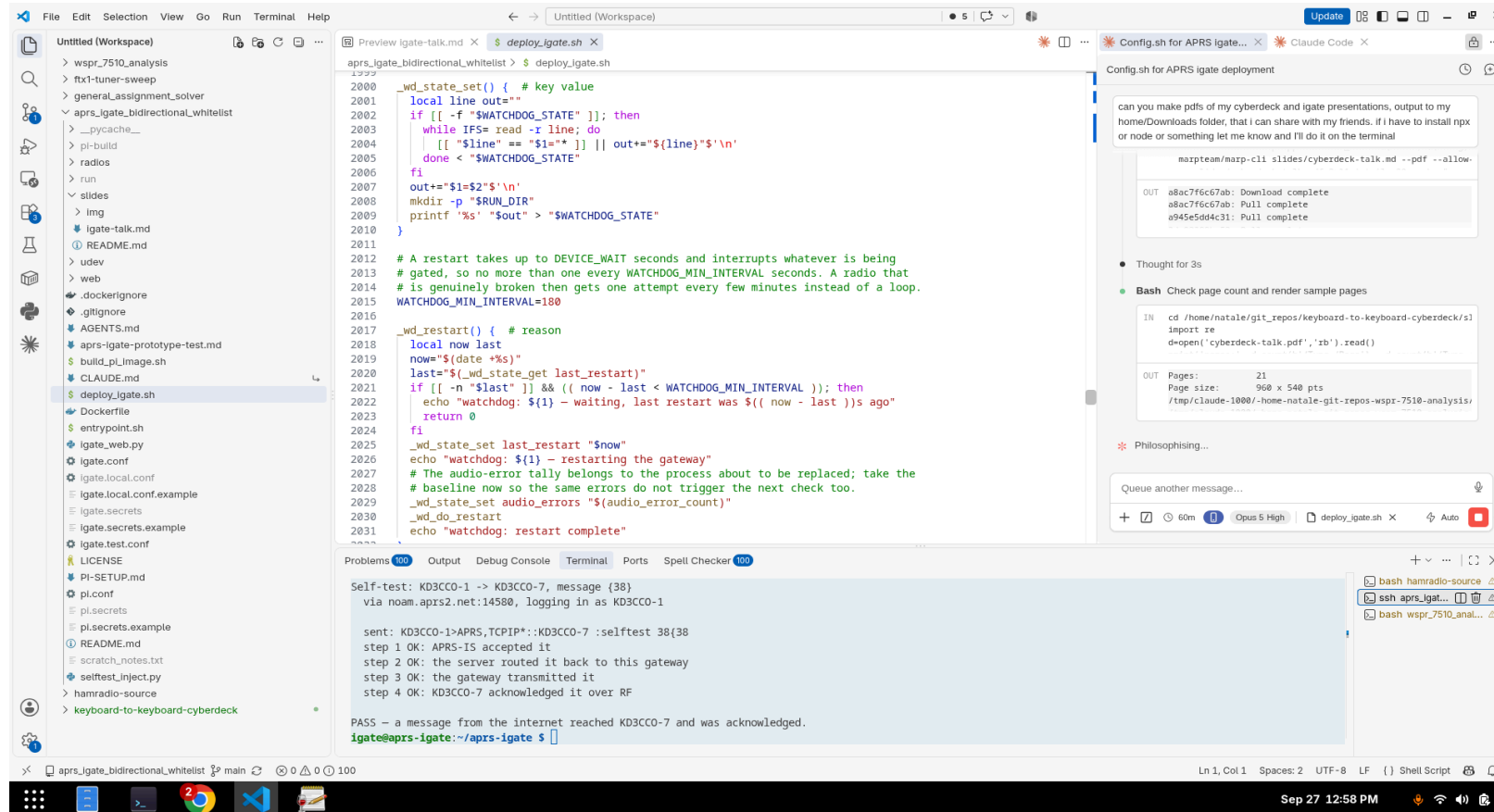
AI helps with faster code and better documentation practices



Hobby time arrives as confetti, so what decides whether a project happens is how much progress fits in twenty minutes. Writing the requirements down first, numbering them, and keeping an honest status used to be the part you skipped; it stops being overhead when the tedium is cheap.

Its best trick is telling me what I did not know to ask — write down how you think something should work, then ask what the alternatives are and what you are not accounting for.

Code, documentation, slides and the blog in one window, where the assistant can see all of it



Documentation is markdown in the repository, beside the code. Everything advances in the same sitting, so nothing drifts. The blog is another repository in the same workspace; these slides are markdown in this one.

A text message to a radio, where there is no cell service

When I am out past coverage with a handheld, somebody at home can reach me, and I can answer — through my own well-behaved gateway

Code, four quickstarts, and a step-by-step Pi build

github.com/hatandspecs/aprs_igate_bidirectional_whitelist

Write-up, including everything that went wrong

hatandspecs.github.io/hamradio/articles/aprs-igate-bidirectional-whitelist/

Runs in Docker on a laptop, or bare-metal on a \$35 Pi.

KD3CCO — questions welcome